- A warning system where exams freeze automatically after five violations.
- Suspicion level scoring to help invigilators assess behavior.
- Reporting features showing activity summaries and student performance.

CheatShield is designed as a web-based solution accessible through modern browsers, ensuring easy scalability and adaptability for future enhancements.

SDLC METHODOLOGIES

The development of **CheatShield** follows the **Agile Software Development Life Cycle (SDLC)** methodology, which emphasizes iterative development, continuous feedback, and flexibility. The project was divided into multiple sprints, each focusing on specific functionalities such as login modules, event tracking, the warning system, and dashboard integration. After every iteration, the system was tested, reviewed, and refined based on user and stakeholder feedback. This approach ensured early detection of issues, smooth integration of new features, and better alignment with project goals. Agile methodology was chosen for **CheatShield** because it supports adaptability to changing requirements, promotes collaboration among developers and users, and enables the delivery of a more reliable, secure, and user-friendly online examination system.

- **Understanding Stakeholder Requirements**

  The development process began with detailed discussions involving both students and invigilators to gather precise requirements. Through meetings and feedback sessions, key challenges in existing online exam systems such as cheating, lack of monitoring, and weak timer enforcement were identified. Based on this understanding, the essential features of **CheatShield**, including event tracking, warning mechanisms, and dual dashboards, were defined to ensure the platform met real user need

- **Sprint Planning**

  The project was divided into multiple sprints, each focusing on specific modules. For example, Sprint 1 focused on the login and authentication system, Sprint 2 implemented the exam interface, Sprint 3 added event tracking and warning logic, and Sprint 4 integrated dashboards and reporting features. This structured sprint planning ensured smooth progress and manageable development cycles.

- **Designing the Product**

  Before every sprint, the design phase included creating interface wireframes, database schemas, and workflow diagrams. For **CheatShield**, the focus was on building a clean, user-friendly interface for students and an intuitive dashboard for invigilators, supported by an efficient backend structure to handle exam data and activity logs.

- **Testing**

  Each sprint concluded with manual testing to verify feature functionality, integration between modules, and overall system stability. This involved unit testing of individual components, integration testing of connected modules, and end-to-end validation using test cases. Tools like **Postman** were also used for API testing to ensure reliable data flow between the client and server.

- **Feedback and Preparing for Next Iteration**

  After every sprint, the developed features were demonstrated to team members and the faculty guide for review. Their feedback was recorded and implemented in subsequent iterations to refine the system. This continuous feedback cycle ensured that **CheatShield** evolved steadily into a secure, transparent, and user-oriented online examination platform.

SYSTEM DESIGN

INTRODUCTION

System design is the process of planning and defining the overall structure, architecture, and functionality of the CheatShield platform to meet the specified project requirements. It involves designing various components such as user interfaces, databases, dashboards, and system interactions that ensure secure, transparent, and efficient online examinations.

For *CheatShield*, the system design focuses on creating two main user interfaces one for **students**, providing a controlled and secure exam environment, and another for **invigilators**, offering real-time monitoring and suspicious activity tracking. The design also defines how different modules like event logging, suspicion scoring, and result generation interact seamlessly to maintain fairness and system integrity throughout the examination process.

- **Components of the CheatShield Application:**

  - **Student Dashboard:**
    This is the primary interface for students, where they can view available exams, start tests, and monitor their progress. The dashboard provides real-time feedback, including remaining time and warning notifications for any suspicious activity such as tab switching, right-clicking, or copy-paste attempts.

  - **Invigilator Dashboard:**
    This module serves as the control panel for invigilators, allowing them to monitor all active exams in real-time. It displays each student's progress, warning count, and suspicious activity level (Low, Medium, High). Invigilators can also freeze a student's exam if multiple violations occur.

- **Authentication Module:**

  This component manages secure login and registration for both students and invigilators. Role-based access ensures that users can only access features relevant to their roles, maintaining data integrity and privacy.

- **Event Logging & Suspicion Scoring:**

  This core module tracks all exam-related activities such as tab changes, copy-paste attempts, or exiting full screen. Each event is logged with a timestamp, and a weighted suspicion score is calculated to determine the overall risk level for each student.

- **Reports & Analytics:**

  After each exam, the system generates detailed reports showing student performance, number of warnings, and suspicious behavior trends. These reports help invigilators assess exam fairness and identify potential malpractice.

Functional Requirements

The functional requirements describe the core operations and expected features that the CheatShield system must provide to its end users students, invigilators, and administrators. These functions define how the system behaves and interacts during an online examination.

- **User Authentication:**

  Both students and invigilators must log in securely using unique credentials. Role-based authentication ensures that each user accesses only their respective dashboard and functionalities.

- **Student Dashboard:**

Students can view available exams, start tests, and attempt questions within a secure interface. The system automatically prevents cheating actions like tab switching, copy-paste, and right-clicks, issuing warnings upon violations.

- **Invigilator Dashboard:**
  Invigilators can monitor all active exams in real-time, view each student's progress, suspicious activity level, and warning count. They can also freeze or terminate a student's exam after repeated violations.

- **Event Tracking and Suspicion Scoring:**
  The system continuously logs suspicious events (tab change, full-screen exit, copy-paste attempt) with timestamps. It calculates a *Suspicion Score* to classify student behavior as Low, Medium, or High risk..

- **Exam Timer and Auto Submission:**
  Each exam is time-bound with an automatic submission feature when time runs out or after multiple violations. This ensures exam integrity and prevents time manipulation.

- **Reporting and Analysis:**
  After exams, detailed reports are generated for invigilators, showing student activity logs, suspicion scores, and warning details.

Non-Functional Requirements

The non-functional requirements define the performance, security, and reliability standards that the **CheatShield** system must maintain. These describe *how* the system operates rather than *what* it does

- **Encryption:**

All sensitive data, including user credentials and exam/session information, is encrypted using advanced algorithms such as **AES-256** and **PBKDF2**, ensuring maximum protection against unauthorized access and cyberattacks.

- **Portability:**

The application is compatible across various operating systems and browsers, including **Windows, macOS, Chrome, Edge, and Firefox**, ensuring flexibility for users and administrators.

- **Reliability:**

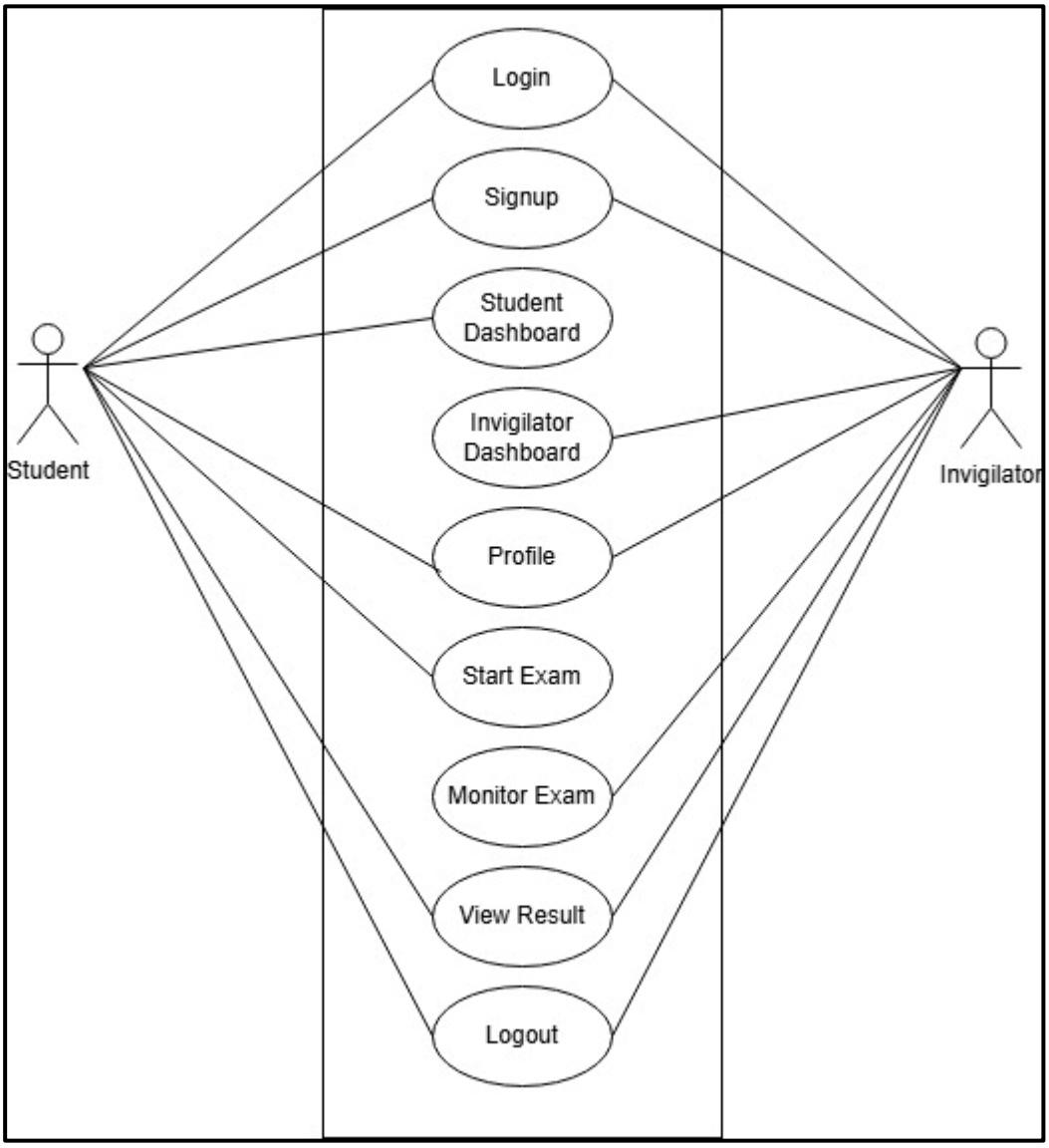
The system ensures continuous availability during online tests and monitoring sessions, minimizing downtime through robust server management and automatic recovery features.

- **Performance:**

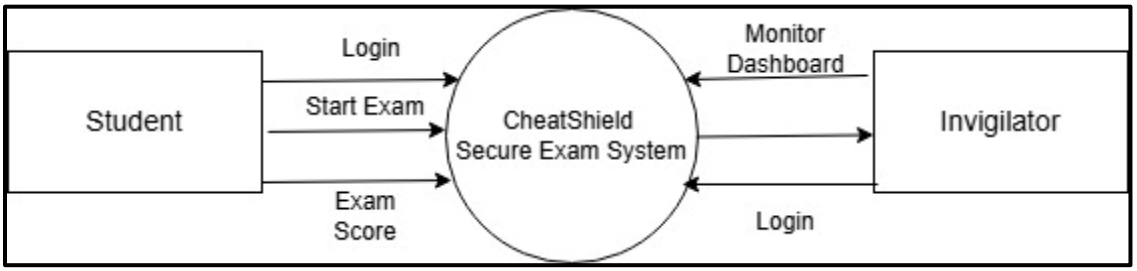
The system ensures minimal latency in real-time monitoring, screen capture, and alert generation, providing a seamless experience for both examiners and candidates.

PHYSICAL DESIGN

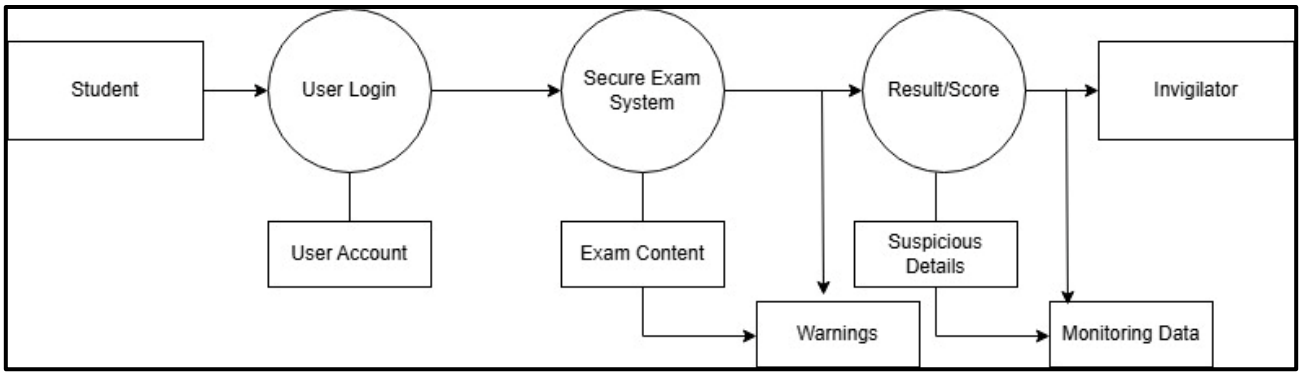
- **Use Case Diagram**



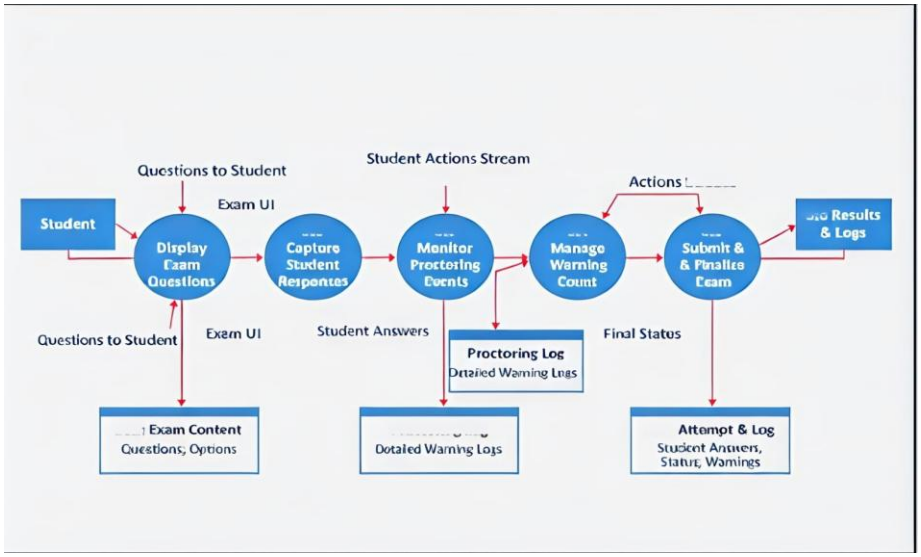
- **Data Flow Diagram – Level 0**



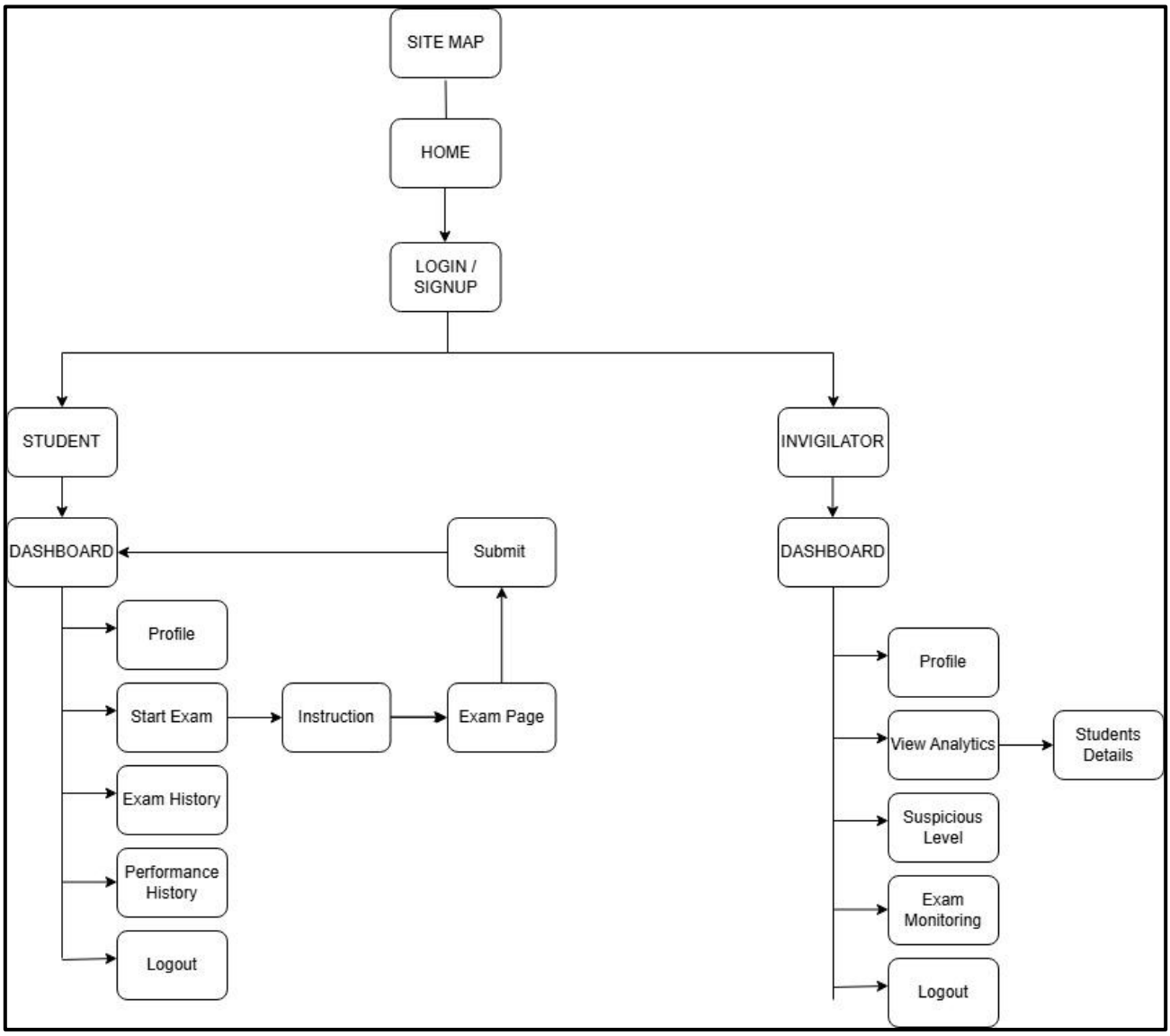
- **Data Flow Diagram – Level 1**



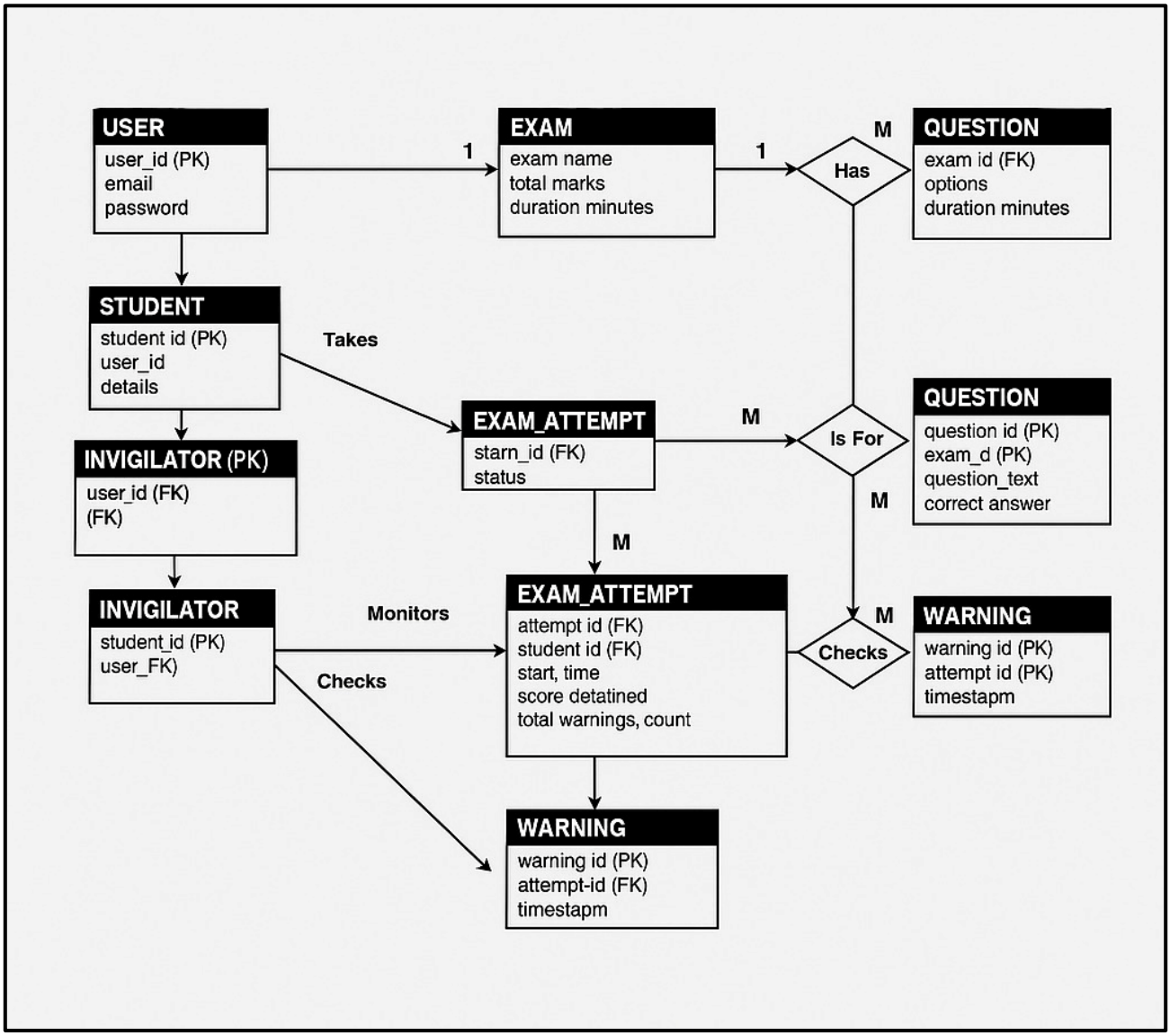
- **Data Flow Diagram – Level 2**



- **Site Map**


SITE MAP
HOME
LOGIN / SIGNUP
STUDENT
INVIGILATOR
DASHBOARD
Submit
DASHBOARD
Profile
Start Exam
Instruction
Exam Page
Exam History
Performance History
Logout
Profile
View Analytics
Students Details
Suspicious Level
Exam Monitoring
Logout

- **ER Diagram**


USER
user_id (PK)
email
password
1
EXAM
exam name
total marks
duration minutes
1
M
Has
QUESTION
exam id (FK)
options
duration minutes
STUDENT
student id (PK)
user_id
details
Takes
EXAM_ATTEMPT
starn_id (FK)
status
M
Is For
QUESTION
question id (PK)
exam_d (PK)
question_text
correct answer
M
INVIGILATOR (PK)
user_id (FK)
(FK)
M
INVIGILATOR
student_id (PK)
user_FK)
Monitors
Checks
EXAM_ATTEMPT
attempt id (FK)
student id (FK)
start, time
score detatined
total warnings, count
M
Checks
WARNING
warning id (PK)
attempt id (PK)
timestapm
WARNING
warning id (PK)
attempt-id (FK)
timestapm

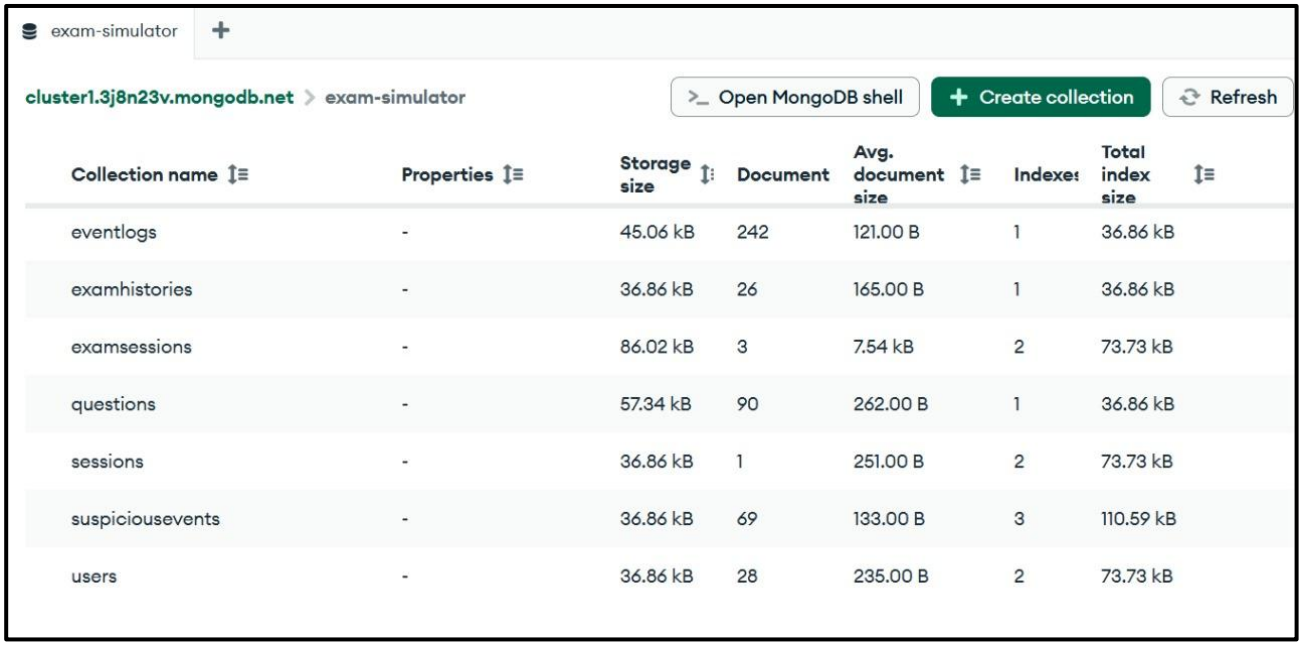
DATABASE DESIGN

The database used in this project is **MongoDB**, an open-source, document-oriented NoSQL database that provides high performance, scalability, and flexibility for data storage. MongoDB stores data in JSON-like documents, making it ideal for handling dynamic and unstructured data efficiently.

For this project, the database is hosted on **MongoDB Atlas**, a fully managed cloud-based database service. MongoDB Atlas simplifies the process of deploying, managing, and scaling databases by automating infrastructure management, performance optimization, and backup operations. It eliminates the need for manual server configuration and maintenance, enabling developers to focus on application development.

Through MongoDB Atlas, the database for CheatShield can be deployed seamlessly on leading cloud service providers such as **AWS**, **Azure**, or **Google Cloud Platform (GCP)**. The use of Atlas ensures enhanced reliability, data security, and automatic failover support, which are crucial for maintaining continuous availability of the system.

The integration of MongoDB in CheatShield allows for efficient management of user data, authentication records, and system logs, ensuring quick data retrieval and smooth operation of the application.



Collection name	Properties	Storage size	Document	Avg. document size	Indexes	Total index size
eventlogs	-	45.06 kB	242	121.00 B	1	36.86 kB
examhistories	-	36.86 kB	26	165.00 B	1	36.86 kB
examsessions	-	86.02 kB	3	7.54 kB	2	73.73 kB
questions	-	57.34 kB	90	262.00 B	1	36.86 kB
sessions	-	36.86 kB	1	251.00 B	2	73.73 kB
suspiciousevents	-	36.86 kB	69	133.00 B	3	110.59 kB
users	-	36.86 kB	28	235.00 B	2	73.73 kB

DATABASE SCHEMA

MongoDB schema design differs significantly from traditional relational database schema design. Unlike SQL databases, MongoDB does not require a predefined structure or formalized schema, allowing greater flexibility in data organization. In MongoDB, data is stored in the form of documents within collections, rather than rows within tables. This flexible schema approach makes it easier to accommodate changes in data models as the project evolves.

In the **CheatShield** project, **Mongoose** (an Object Data Modeling library for MongoDB and Node.js) is used to define and manage data schemas. Mongoose provides a structured way to model application data while leveraging MongoDB's flexibility.

Sample database schema implemented in the project:

```
const userData= new mongoose.Schema({
      name: {type:String, required:true},
      email: {type:String, required:true, unique:true},
      password: {type:String, required:true},
      mobileNo: {type:Number, required:true},
      role: {type:String, enum:["Student","Invigilator"], required:true},
      status: {type:String, default:"active"},
      warnings: {type:Number, default:0},
      suspicionscore: {type:Number, default:0},
});
module.exports= mongoose.model("User", userData);
```